Movie Ticket Booking System

Design & Architecture Walkthrough

A concurrency-safe seat-booking backend - no seat is ever sold twice

Requirements | HLD | Concurrency | ER Diagram | Tech Stack | Testing | AI Workflow

Java 21 - Spring Boot 3.3 - PostgreSQL + Flyway - JWT

1. Functional Requirements

What the system must do - in plain terms

Core booking journey

- Browse shows by city, movie and date.
- View the live seat map with prices.
- Hold selected seats for a short window.
- Confirm & pay to lock the booking.
- Cancel a booking and get a refund.

Pricing & money

- Tiered seat pricing (Regular / Premium / Recliner).
- One booking = one pricing tier (no mixing).
- Preview & apply discount coupons before paying.
- Policy-based refunds on cancellation.

Correctness & scale

- No seat is ever sold twice (prime invariant).
- Safe under thousands of concurrent requests.
- Unpaid holds expire automatically.
- Every state change is auditable.

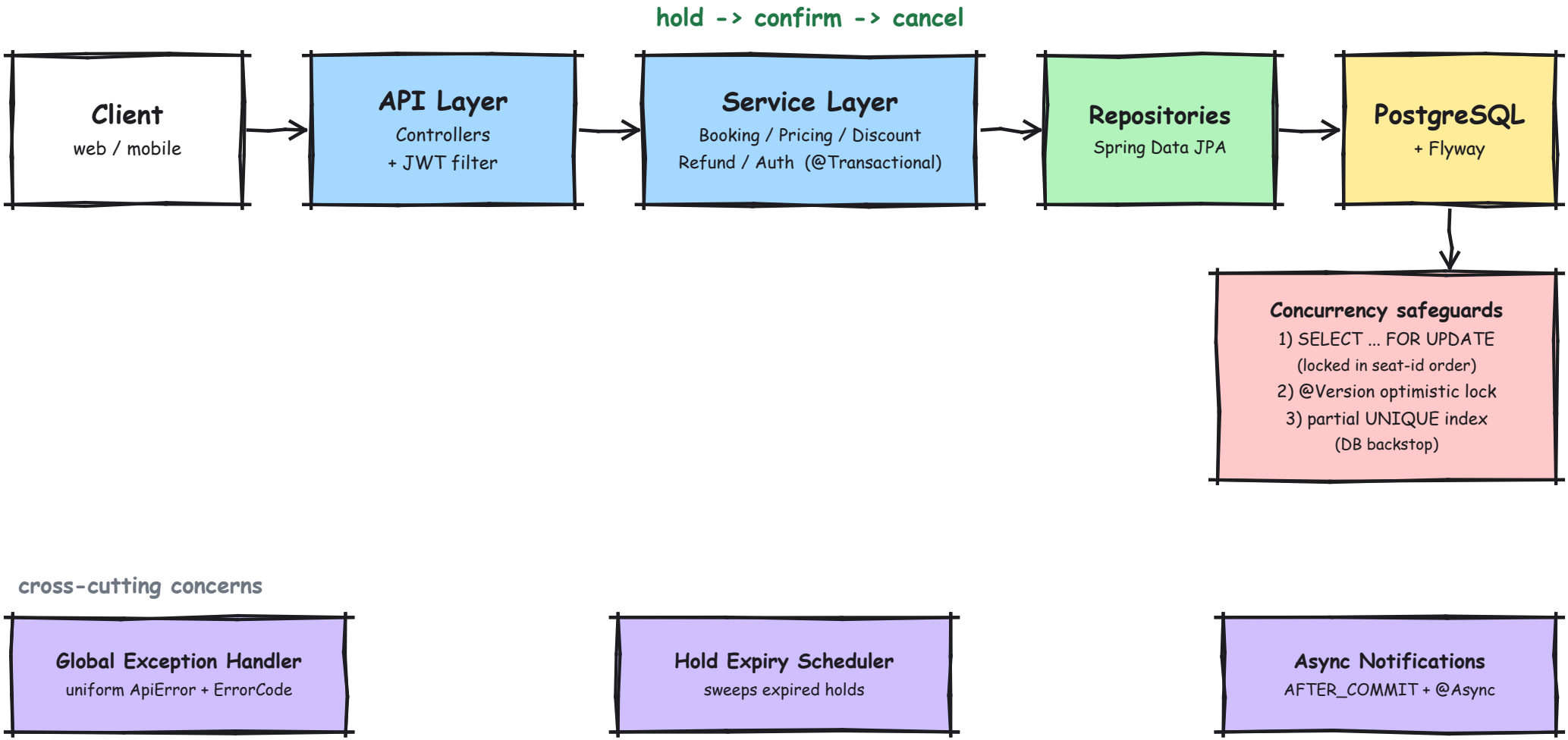
Roles & access

- ADMIN: manage catalog, shows, pricing, discounts.
- CUSTOMER: browse, book, cancel own bookings.
- Stateless JWT authentication.

Prime invariant: NO SEAT IS EVER SOLD TWICE - correctness is prioritized over feature count.

2. High-Level Design

Request flow: thin controllers -> services -> repositories -> PostgreSQL



Controllers are thin; business logic and transaction boundaries live in the service layer. Seat contention is per-row, so unrelated seats/shows proceed fully in parallel.

3. Concurrency: Approach & Trade-offs

Goal: no double-sell, with low latency and simple engineering

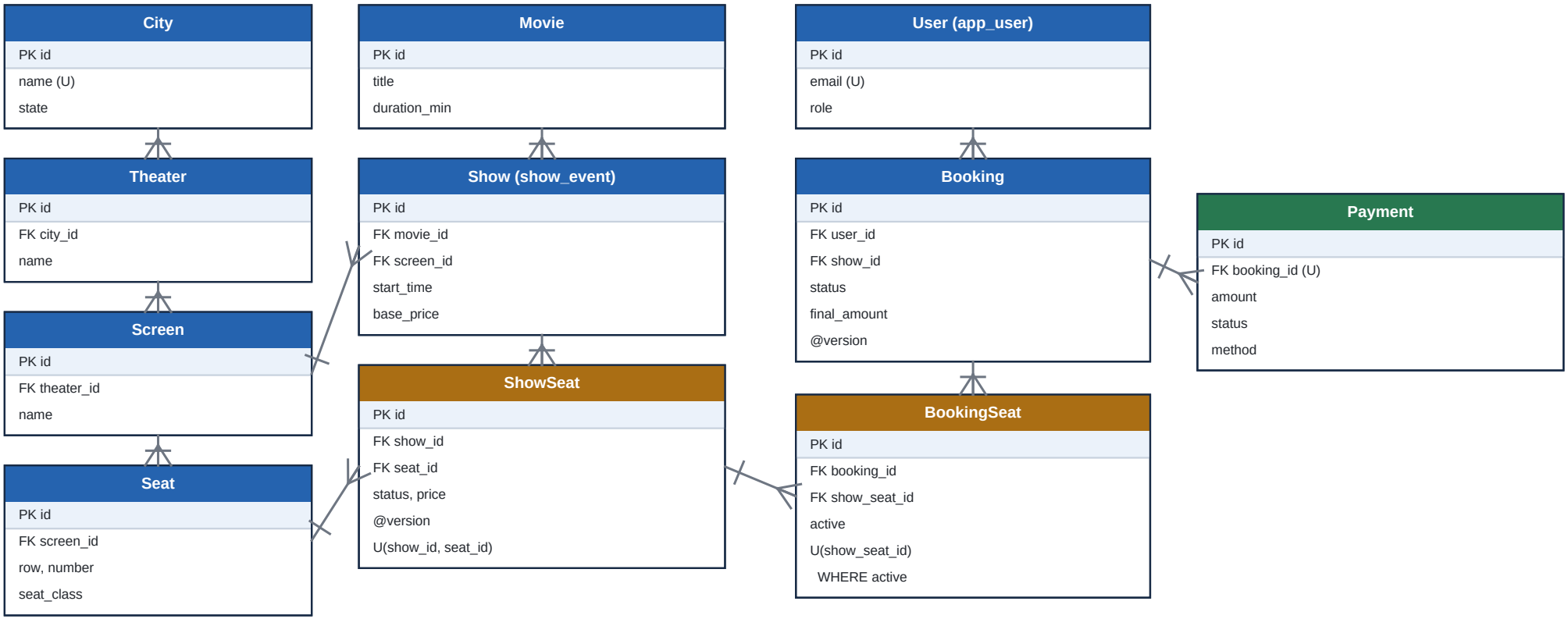
Approach	Latency impact	Engineering effort	Fit for our goal
PostgreSQL row locks (SELECT FOR UPDATE)	- Lowest - synchronous ms answer; blocks only on the same seat row.	- Simplest - native ACID; no extra infra to run or monitor.	**BEST FIT**
Synchronous queue	- High - all bookings serialize through one worker.	- In-memory queue breaks across multiple app nodes.	**Anti-pattern**
Async queue	- Fast 202, but slow real answer (eventual consistency).	- Broker + consumers + idempotency + result delivery.	**Bursts only**
Event-specific queues	- Better globally, but a hot show still serializes.	- Dynamic per-event queues, routing, autoscaling.	**Reinvents DB**
Non-overlapping queues (shard by seat)	- Great parallelism once built correctly.	- Sharding + cross-shard atomicity + rebalancing.	**Too complex**

Verdict: PostgreSQL row-level locking wins.

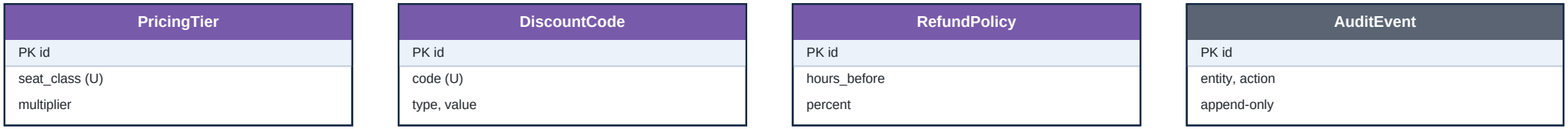
The 'seat taken' decision must be serialized at one point per seat anyway - Postgres does exactly that, synchronously, with an immediate answer and zero extra infrastructure. Queues sit IN FRONT OF the DB, not instead of it: keep them for async side-effects (notifications), never for seat arbitration.

4. ER Diagram

SmartDraw-style physical model: PK / FK attributes with crow's-foot cardinality



Configuration & audit (reference data - no FK into the booking flow)



PK=primary key FK=foreign key
(U)=unique ->|< = crow's foot 'many'

5. Tech Stack & Reasoning

Every choice serves: correctness, low latency, simple engineering

Java 21 + Spring Boot 3.3

Mature ecosystem; declarative @Transactional, DI, and security out of the box.

PostgreSQL

ACID + row-level locking + partial indexes = the concurrency backbone. No double-sell.

Flyway migrations

Versioned, authoritative schema; app runs ddl-auto=validate (never auto-mutates DB).

JWT auth (ADMIN / CUSTOMER)

Stateless tokens -> horizontal scaling, no server session store.

Spring Data JPA + Hibernate

Repositories + @Version; pessimistic lock via a simple derived query.

Zonky embedded PostgreSQL

Real Postgres in tests (true locking) without a Docker daemon.

Bean Validation + DTOs

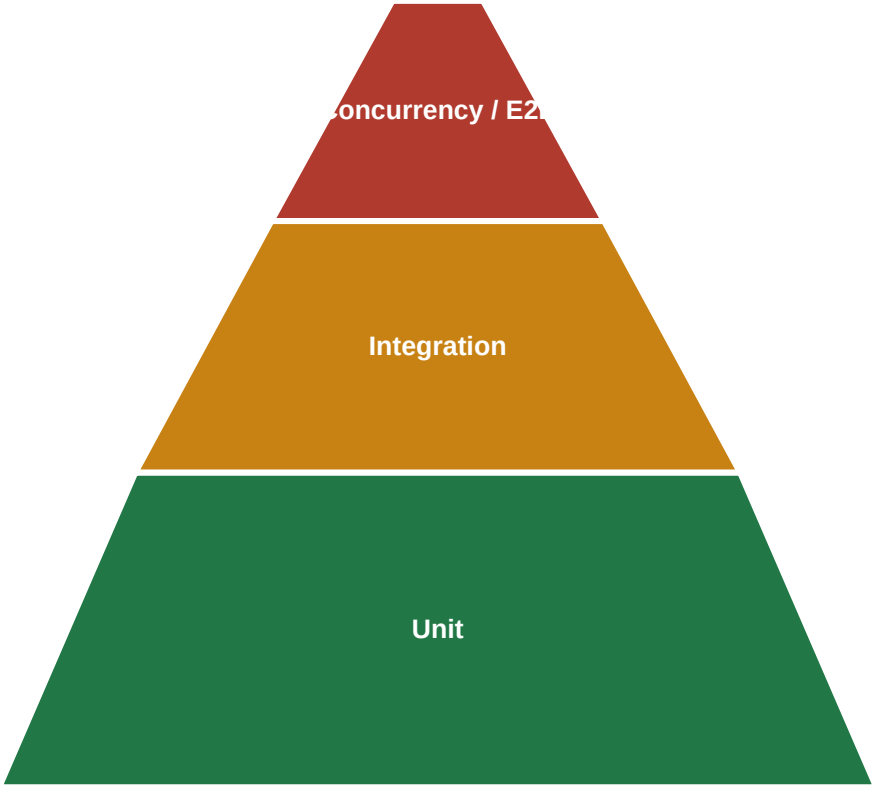
Reject bad input at the edge; entities never leak out of controllers.

Maven + JUnit 5 / Mockito

Standard build; fast unit tests plus full integration + concurrency suite.

6. Testing Approach

Correctness is demonstrated by execution, not asserted



Fewer, high-value tests at the top

Concurrency race (flagship)

- SeatBookingConcurrencyTest: 20 threads, 1 seat.
- Asserts exactly ONE winner, seat ends BOOKED.
- Proves the no-double-sell invariant under load.

Integration (@IntegrationTest + Zonky)

- Real embedded PostgreSQL - true locking semantics.
- Lifecycle, hold-expiry, RBAC, discount preview, single-tier rule.

Unit (Mockito, no Spring)

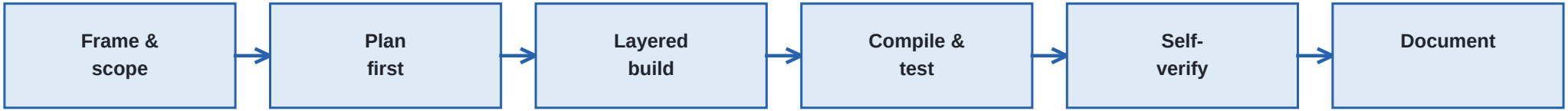
- Pricing, discount, and refund pure logic.
- Fast feedback on business rules.

One command: mvn test

Runs the whole pyramid, including the 20-thread race against a real Postgres. Test IDs are traced in the master test-case document.

7. AI Workflow

How an AI agent (Cursor) was used - with humans in the loop



How the agent was directed

- A single detailed build-prompt fixed the stack, domain, API and concurrency defense.
- AGENTS.md = durable guardrails the agent must respect on every change.
- Concurrency rules explicitly 'do not weaken'.
- Tight, testable acceptance criteria kept it honest.

Verification loop

- Explore -> propose -> edit -> compile -> run tests.
- Self-check artifacts (rendered PDFs, test output).
- Fixed real issues (Zonky provider, SQL null-type).

Produced by AI

- Entities, Flyway migrations, services, controllers.
- Security, DTOs, exception handling.
- Unit + integration + concurrency tests, and docs.

Owned by humans

- Architecture decisions and trade-off calls.
- Code review of every change; small focused commits.
- Correctness demonstrated by execution, not trust.

Principle: AI accelerates delivery; the 20-thread race test - not the model - proves correctness.